



MCP Client — Quick Revision Notes

1. MCP Playlist Structure

- **WHY** → Why MCP is needed
- **WHAT** → Architecture + Lifecycle
- **HOW** → Using MCP, building servers (local + remote)

This video = final piece → **How to build an MCP Client.**

2. Goal of the Video

Build a **custom MCP chatbot** that:

- Can chat normally
 - Can connect to multiple MCP servers
 - Supports:
 - **Local Math server**
 - **Remote Expense Tracker server**
 - **Manim animation server**
 - Works through a **Streamlit GUI**
-

3. Libraries Used

- `langchain`
 - `langchain-openai`
 - `langchain-mcp-adapters`
 - `streamlit`
 - `uv` (uv tool & uvicorn)
 - Python (async/await)
-

4. Local Math Server Setup

- Math server includes tools: `add`, `subtract`, `multiply`, `divide`, `power`, `modulus`
 - Tested using:
 - `uv run fastmcp dev main.py`
 - MCP Inspector shows tools working
-

5. Building the MCP Client (Core Steps)

Step 1 → Server Config

Define server in a dictionary:

- Name
- Transport (`stdio`)
- Command to start the server (`uv run fastmcp run main.py`)

Step 2 → Create MCP Client

```
client = MultiServerMCPClient(servers=servers)
```

Step 3 → Fetch Tools

```
tools = await client.get_tools()
```

Convert list → dict `{tool_name: tool_obj}`

6. Bind LLM With Tools

```
llm = ChatOpenAI(model="gpt-5")  
llm_with_tools = llm.bind_tools(tools)
```

7. Tool-Call Flow (Super Important)

1) Send prompt → Receive response

LLM may return:

- **Content** (normal answer), OR
- **Tool call** (with name + args)

2) Extract tool call

```
tool_name = response.tool_calls[0].name
args = response.tool_calls[0].args
id = response.tool_calls[0].id
```

3) Execute tool

```
result = await named_tools[tool_name].ainvoke(args)
```

4) Send result back to LLM

Create `ToolMessage` with:

- `id`
- `content = result`

LLM returns the **final human-readable answer**.

8. Handle Normal Chat

- If `response.tool_calls` does **not** exist → just display `response.content`.
-

9. Multi-Tool / Multi-Server Support

LLM may call multiple tools → handle using a **loop**:

- For each tool call → extract → execute → push ToolMessage
-

10. Adding Additional Servers

A) Remote Expense Tracker

Transport: `streamable_http`

Requires only:

- Server name
- URL from `fastmcp.cloud`

Tools added:

- `add_expense`
- `list_expenses`
- `summarize`

B) Manim Server

Transport: `stdio`

Used to generate animation videos via MCP.

11. Building the GUI (Streamlit)

- All logic reused
- Streamlit handles:
 - User prompts
 - Chat window
 - Displaying lists (expenses)
 - Showing generated animations

Run using:

```
streamlit run client2.py
```

-
-

12. End Results

You now have:

- A working **MCP chatbot client**
- Connected to **3 servers simultaneously**
- A clean **Streamlit UI**
- Ability to add *any* MCP server from the internet